

PATRÓN DE DISEÑO: COMPUESTO (COMPOSITE)

Capítulo 15 — Patrones de Diseño en Java

Traducción técnica al español

© 2004 CRC Press LLC — Traducción al español

1. Descripción

Este patrón fue descrito previamente en GoF95.

Todo componente u objeto puede clasificarse en una de dos categorías: Componentes Individuales o Componentes Compuestos, los cuales están conformados por componentes individuales u otros componentes compuestos. El Patrón Compuesto es útil para diseñar una interfaz común tanto para componentes individuales como para componentes compuestos, de modo que los programas cliente puedan ver ambos tipos de manera uniforme. En otras palabras, el Patrón Compuesto permite que un objeto cliente trate tanto los componentes individuales como los compuestos de manera idéntica.

Esto también puede explicarse en términos de una estructura de árbol. El Patrón Compuesto permite la referencia uniforme tanto a nodos no terminales (que representan grupos de componentes o compuestos) como a nodos terminales (que representan componentes individuales).

2. Ejemplo — Simulación del Sistema de Archivos

Crearemos una aplicación para simular el sistema de archivos de Windows/UNIX. Un sistema de archivos consiste principalmente en dos tipos de componentes: directorios y archivos. Los directorios pueden estar compuestos por otros directorios o archivos, mientras que los archivos no pueden contener ningún otro componente del sistema de archivos. En este aspecto, los directorios actúan como nodos no terminales y los archivos como nodos terminales de una estructura de árbol.

3. Enfoque de Diseño I

Se define una interfaz común para directorios y archivos mediante la interfaz Java `FileSystemComponent`. La interfaz `FileSystemComponent` declara los métodos comunes tanto para componentes de archivo como para componentes de directorio.

Se definen además dos clases — `FileComponent` y `DirComponent` — como implementadoras de la interfaz común `FileSystemComponent`.

3.1 FileComponent

La clase `FileComponent` representa un archivo en el sistema de archivos e implementa el siguiente método:

`getComponentSize()`

Este método retorna el tamaño (en kilobytes) del archivo representado por el objeto `FileComponent`.

3.2 DirComponent

Esta clase representa un directorio en el sistema de archivos. Dado que los directorios son entidades compuestas, el `DirComponent` provee métodos para gestionar el contenido que contiene. Estos métodos son adicionales al método común `getComponentSize()` declarado en la interfaz `FileSystemComponent`.

Tabla 1 — Métodos de `DirComponent`

Método	Descripción
<code>addComponent(FileSystemComponent)</code>	Usado por aplicaciones cliente para agregar objetos <code>DirComponent</code> o <code>FileComponent</code> a un <code>DirComponent</code> .
<code>getComponent(int)</code>	El <code>DirComponent</code> almacena otros objetos <code>FileSystemComponent</code> en un vector. Este método recupera uno de esos objetos almacenado en la posición indicada.
<code>getComponentSize()</code>	Retorna el tamaño (en kilobytes) del directorio representado por el objeto <code>DirComponent</code> . Itera recursivamente sobre la colección de <code>FileSystemComponent</code> que contiene y suma los tamaños de todos los <code>FileComponent</code> individuales.

Un cliente típico crearía primero un conjunto de objetos `FileSystemComponent` (instancias de `DirComponent` y `FileComponent`). Puede usar el método `addComponent` del `DirComponent` para agregar diferentes `FileSystemComponent` a un `DirComponent`, creando una jerarquía de objetos del sistema de archivos.

Cuando el cliente desea consultar el tamaño de cualquiera de estos objetos, puede invocar el método `getComponentSize`. El cliente no necesita conocer los cálculos involucrados ni la forma en que se realizan para determinar el tamaño del componente. En este aspecto, el cliente trata

al objeto `FileComponent` y al objeto `DirComponent` de la misma manera — no se requiere código separado para consultar el tamaño de objetos `FileComponent` y objetos `DirComponent`.

Sin embargo, aunque el cliente trate ambos objetos de manera uniforme para el método común `getComponentSize`, sí necesita distinguir al invocar métodos específicos del compuesto, como `addComponent` y `getComponent`, definidos exclusivamente en `DirComponent`. Dado que estos métodos no están disponibles en `FileComponent`, el cliente necesita verificar que el `FileSystemComponent` con el que trabaja es efectivamente un objeto `DirComponent`.

El siguiente Enfoque de Diseño II elimina este requisito del cliente.

4. Enfoque de Diseño II

El objetivo de este enfoque es:

- Proveer la misma ventaja de permitir que la aplicación cliente trate al `DirComponent` compuesto y al `FileComponent` individual de manera uniforme al invocar el método `getComponentSize`.
- Liberar a la aplicación cliente de tener que verificar si el `FileSystemComponent` con el que trabaja es una instancia de la clase `DirComponent` al invocar cualquiera de los métodos específicos del compuesto, como `addComponent` o `getComponent`.

En el nuevo diseño, los métodos específicos del compuesto `addComponent` y `getComponent` se trasladan a la interfaz común `FileSystemComponent`. El `FileSystemComponent` provee la implementación por defecto para estos métodos y se diseña como clase abstracta.

La implementación por defecto de estos métodos corresponde a lo que aplica a los objetos `FileComponent`. Los objetos `FileComponent` son componentes individuales y no contienen otros objetos `FileSystemComponent` dentro. Por lo tanto, la implementación por defecto no hace nada y simplemente lanza una excepción personalizada `CompositeException`. La clase derivada compuesta `DirComponent` sobrescribe estos métodos para proveer una implementación personalizada.

Dado que no hay cambios en la forma en que se diseña el método común `getComponentSize`, el cliente seguirá pudiendo tratar tanto al `DirComponent` como al `FileComponent` de manera idéntica.

Dado que la clase padre común `FileSystemComponent` ahora provee implementaciones por defecto para los métodos `addComponent` y `getComponent`, la aplicación cliente no necesita hacer ninguna verificación antes de llamar a estos métodos específicos del compuesto.

Cada vez que se necesita agregar un nuevo método específico del compuesto, como `removeComponent`, al `DirComponent` compuesto, también debe agregarse a la clase padre `FileSystemComponent`. El `FileSystemComponent` debe proveer una implementación por defecto específica para `FileComponent` en ese método.

5. Implementación

Listado 5.1 — Clase Abstracta FileSystemComponent

```
public abstract class FileSystemComponent {
    String name;

    public FileSystemComponent(String cName) {
        name = cName;
    }

    public void addComponent(FileSystemComponent comp)
        throws CompositeException {
        throw new CompositeException(
            "Operacion invalida. No soportada");
    }

    public FileSystemComponent getComponent(int componentLocation)
        throws CompositeException {
        throw new CompositeException(
            "Operacion invalida. No soportada");
    }

    public abstract long getComponentSize();
} // fin de la clase FileSystemComponent
```

Listado 5.2 — Subclases Concretas: FileComponent y DirComponent

```
public class FileComponent extends FileSystemComponent {
    private long size;

    public FileComponent(String cName, long sz) {
        super(cName);
        size = sz;
    }

    public long getComponentSize() {
        return size;
    }
} // fin de la clase

public class DirComponent extends FileSystemComponent {
    Vector dirContents = new Vector();
    // coleccion de archivos individuales / subcarpetas

    public DirComponent(String cName) {
        super(cName);
    }

    public void addComponent(FileSystemComponent fc)
```

```

        throws CompositeException {
            dirContents.add(fc);
        }

    public FileSystemComponent getComponent(int location)
        throws CompositeException {
        return (FileSystemComponent) dirContents.elementAt(location);
    }

    public long getComponentSize() {
        long sizeOfAllFiles = 0;
        Enumeration e = dirContents.elements();
        while (e.hasMoreElements()) {
            FileSystemComponent component =
                (FileSystemComponent) e.nextElement();
            sizeOfAllFiles += component.getComponentSize();
        }
        return sizeOfAllFiles;
    }
} // fin de la clase

```

Listado 5.3 — Clase Cliente CompositeDemo

```

public class CompositeDemo {
    public static final String SEPARATOR = ", ";

    public static void main(String[] args) {
        FileSystemComponent mainFolder =
            new DirComponent("Year2000");
        FileSystemComponent subFolder1 =
            new DirComponent("January");
        FileSystemComponent subFolder2 =
            new DirComponent("February");
        FileSystemComponent folder1File1 =
            new FileComponent("Jan1DataFile.txt", 1000);
        FileSystemComponent folder1File2 =
            new FileComponent("Jan2DataFile.txt", 2000);
        FileSystemComponent folder2File1 =
            new FileComponent("Feb1DataFile.txt", 3000);
        FileSystemComponent folder2File2 =
            new FileComponent("Feb2DataFile.txt", 4000);

        try {
            mainFolder.addComponent(subFolder1);
            mainFolder.addComponent(subFolder2);
            subFolder1.addComponent(folder1File1);
            subFolder1.addComponent(folder1File2);
            subFolder2.addComponent(folder2File1);
            subFolder2.addComponent(folder2File2);
        } catch (CompositeException ex) { }

        // El cliente referencia componentes compuestos
    }
}

```

```

        // e individuales de manera uniforme
        System.out.println(" Main Folder Size= " +
            mainFolder.getComponentSize() + "kb");
        System.out.println(" Sub Folder1 Size= " +
            subFolder1.getComponentSize() + "kb");
        System.out.println(" File1 in Folder1 Size= " +
            folder1File1.getComponentSize() + "kb");
    }
}

```

Cuando el cliente CompositeDemo se ejecuta, se muestra la siguiente salida:

```

Main Folder Size= 10000kb
Sub Folder1 Size= 3000kb
File1 in Folder1 Size= 1000kb

```

6. Comparación de Enfoques de Diseño

Tabla 2 — Comparación entre Enfoque I y Enfoque II

Aspecto	Enfoque I	Enfoque II
Interfaz común	Interfaz Java (FileSystemComponent)	Clase abstracta (FileSystemComponent)
Métodos compuestos	Solo en DirComponent	En FileSystemComponent (con impl. por defecto) y sobreescritos en DirComponent
Verificación de tipo por el cliente	Necesaria antes de llamar addComponent / getComponent	No necesaria: la clase base lanza excepción por defecto
Manejo de errores	El cliente debe verificar el tipo	CompositeException lanzada automáticamente si FileComponent recibe llamada compuesta
Extensibilidad	Agregar método compuesto requiere cambio solo en DirComponent	Agregar método compuesto requiere actualizar FileSystemComponent y DirComponent

7. Preguntas de Práctica

1. El siguiente es un ejemplo de un FQDN (nombre de dominio completamente calificado) en Internet:

```
nwest.sales.DomainName.com
```

Está compuesto por diferentes subdominios. Cada subdominio puede mapearse a un directorio específico en el sistema de archivos del computador donde está alojado DomainName.com. Cada subdominio puede tener archivos HTML, accesibles mediante una URL. Así, un subdominio y el conjunto de archivos HTML pueden verse como dos componentes principales en el sistema.

- a. Defina una jerarquía de subdominios para un dominio de ejemplo.
 - b. Cree una aplicación que utilice el Patrón Compuesto para:
 - i. Mostrar el directorio al que está mapeado un subdominio dado.
 - ii. Mostrar las URLs de los componentes del sitio web (ya sean subdominios o archivos HTML individuales) de manera uniforme.
2. Consideremos la etiqueta HTML <frameset>. La etiqueta <frameset> se utiliza para dividir una página web en diferentes secciones con el fin de mostrar múltiples páginas web. Cada sección puede mostrar una página web separada. La página web real que se mostrará dentro de una sección puede especificarse usando la etiqueta <frame>. Además, una etiqueta <frameset> puede anidarse, permitiendo una mayor división de esas secciones. Ejemplo:

```
<html>
  <head></head>
  <frameset rows="20%,80%">
    <frameset rows="100,200">
      <frame src="frame1.html">
      <frame src="frame2.html">
    </frameset>
    <frame src="frame3.html">
  </frameset>
</html>
```

De la descripción de las etiquetas <frameset> y <frame> puede observarse que pueden organizarse en una estructura de árbol, con la etiqueta <frameset> como nodo no terminal y cada etiqueta <frame> como nodo terminal.

- c. Diseñe dos clases — FrameSet y Frame — para representar las etiquetas <frameset> y <frame>, respectivamente.
 - d. Defina una operación getSourceFiles() en estas clases para retornar los archivos HTML especificados para ser mostrados por un objeto FrameSet o Frame específico.
 - e. Diseñe e implemente esta operación aplicando el Patrón Compuesto, de modo que un cliente pueda referirse a estas clases de manera idéntica.
3. Una base de datos típica de productos consiste en dos tipos de componentes de productos: categorías de productos y artículos de productos. Una categoría de producto es compuesta por naturaleza. Puede contener artículos de productos y también otras categorías de productos como subcategorías. Categorías de productos de ejemplo:
- f. Computadoras

- g. Escritorios
- h. Portátiles
- i. Periféricos
- j. Impresoras
- k. Cables

La categoría de productos Computadoras contiene tanto la categoría Escritorios como la categoría Portátiles como subcategorías. La categoría Escritorios puede contener un artículo de producto como Compaq Presario 5050. Los artículos de productos son generalmente individuales, en el sentido de que no contienen ningún componente de producto dentro de ellos.

Diseñe e implemente una aplicación para listar el valor en dólares de cada componente. Use el Patrón Compuesto para permitir que la aplicación cliente se refiera tanto a las categorías de productos como a los artículos de productos de manera uniforme.